

# Freight Rate Prediction Challenge

Validation Approach, Modeling Report & December Rate Forecast

Candidate submission - Machine Learning Engineer Assessment

## 1. Data Overview & Key Findings

train-test.csv contains 48,000 labeled loads from 2025-01-01 through 2025-10-31 across 64 US cities, 3 equipment types (Dry Van, Reefer, Flatbed) and a target of posted\_rate (\$). validation.csv (12,000 loads, needing predictions) spans 2025-11-01 through 2025-12-31 and includes 8 cities never seen in training (Knoxville, San Diego, Norfolk, Jackson, Allentown, Charlotte, Chicago, Laredo), which shaped the feature design below. december-chart-inputs.csv holds 31 rows, one per December day, with pickup, delivery, distance, equipment and weight all fixed - only the date changes.

Key relationships found during EDA:

- distance is by far the strongest predictor (Pearson  $r = 0.91$  with posted\_rate).
- Equipment type has a smaller but consistent effect on rate per mile: Reefer > Flatbed > Dry Van.
- Rate per mile shows genuine seasonality: it rises from January into a June peak (~\$2.33/mi) and eases back down through the fall (~\$2.19-2.24/mi), a plausible summer-freight-demand pattern.
- market\_index and quote\_signal have very weak linear correlation with posted\_rate ( $|r| < 0.07$ ) and were confirmed low-importance in model testing - see Section 3.

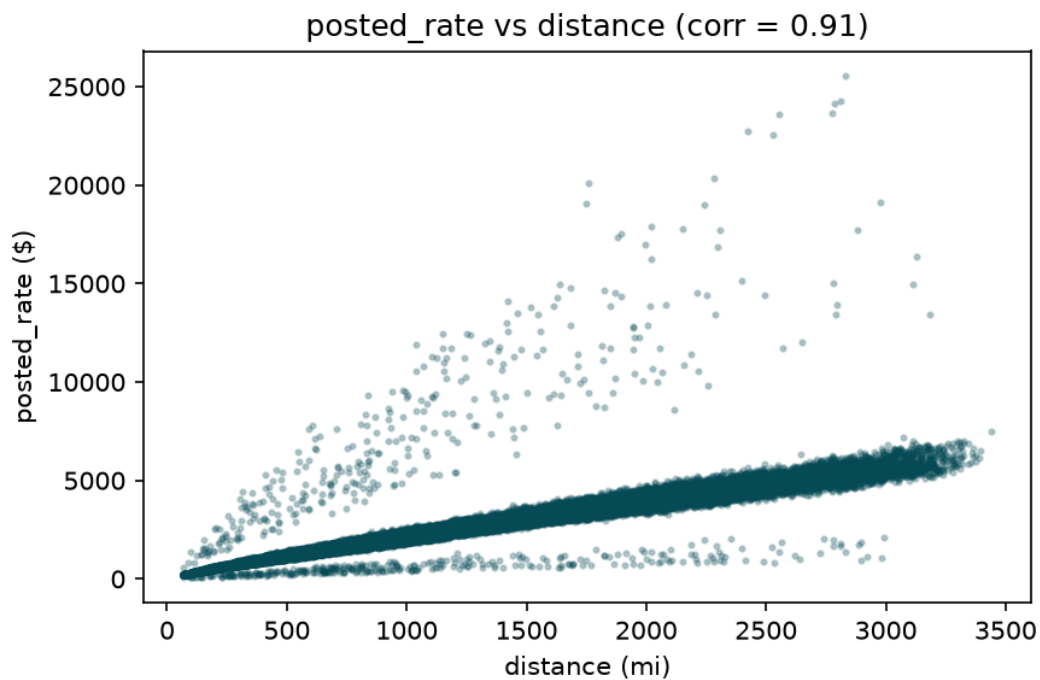


Figure 1. posted\_rate vs distance

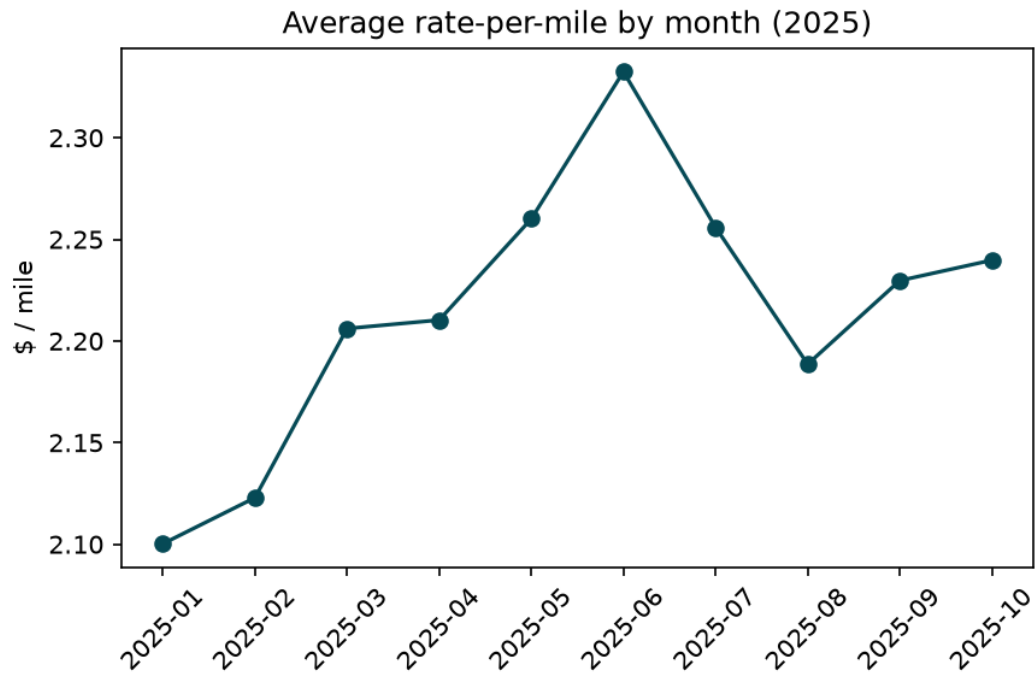


Figure 2. Average rate-per-mile by month

## 2. Data Quality Issues & How They Were Addressed

### 2.1 Sign-flip errors in weight

292 of 48,000 rows (0.6%) had a negative weight. The negated values' distribution is statistically identical to the positive-weight distribution (same mean, std and range), confirming a sign-entry error rather than a distinct population. Fix: take the absolute value (src/features.py: clean\_weight).

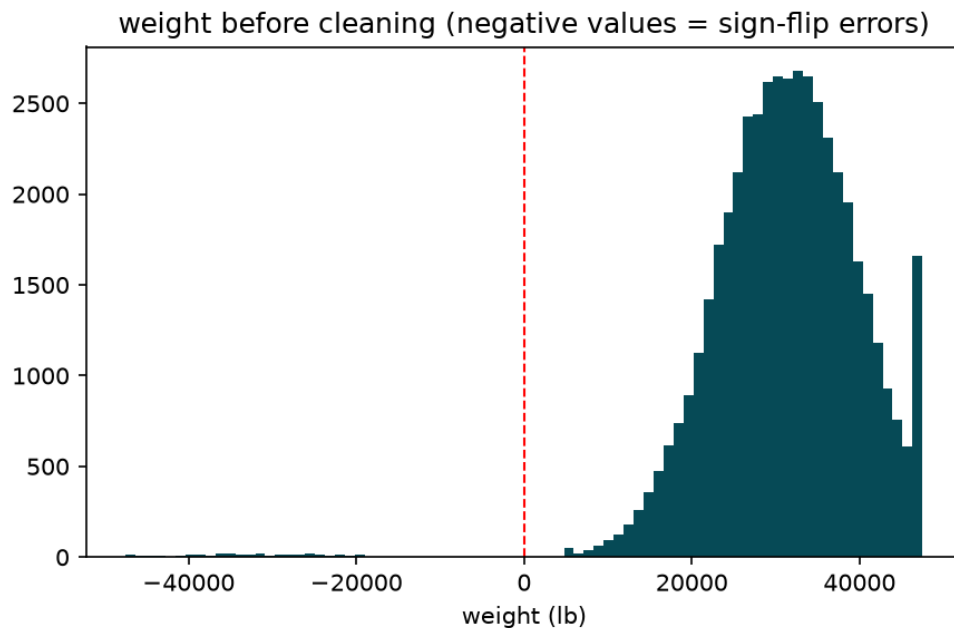


Figure 3. Raw weight distribution before cleaning

### 2.2 Missing values

weight was missing for 300 training rows (0.6%) and 165 validation rows; imputed with the median weight computed on the training split only, so no information leaks across the train/holdout boundary or into validation. market\_index was missing for 374 training rows (0.8%) and 249 validation rows, but since this feature was excluded from the model (Section 3), no imputation was needed for it.

### 2.3 Unseen cities in validation.csv

8 of the 72 cities in validation.csv never appear in training. A model that encodes pickup/delivery as plain categorical IDs (one-hot or target encoding) has no way to score these rows sensibly. Instead, pickup/delivery latitude & longitude are used as continuous features, which generalize smoothly to new cities. A stress test (see Section 3) confirmed this: predictions on routes touching held-out cities were only ~15% worse than for fully-seen routes, rather than failing outright.

### 2.4 Distance is not a fixed lane property

3,944 of 4,014 pickup-delivery city pairs have more than one distinct distance value across loads, i.e. distance varies per shipment (routing, detours) rather than being a lookup constant. This is expected and was used per-row, not averaged per lane.

### 2.5 Unexplained high-rate outliers

~0.7% of loads have a rate-per-mile more than double the typical range (\$10-14/mi vs. a ~\$2/mi median), with no correlation to equipment, month, market\_index, quote\_signal or negative-weight flags that were tested. These look like genuine spot-market rate spikes (e.g. capacity crunches) that are not explainable from the given features, so they were kept in training rather than dropped as errors, and error metrics were evaluated with both MAE/MAPE (robust) and RMSE (sensitive to this tail) for transparency.

## 3. Feature Engineering & Model Selection

### 3.1 Features used

distance, weight (cleaned), pickup/delivery lat & lon, equipment (one-hot), and date decomposed into cyclical sin/cos encodings for month, day-of-year and day-of-week plus an is\_weekend flag. market\_index and quote\_signal were deliberately excluded: they are absent from december-chart-inputs.csv, and an ablation showed they change holdout MAE by less than 1%, so dropping them keeps one consistent pipeline for both the validation and December predictions instead of maintaining a second model or an imputation scheme for missing market signals in December.

### 3.2 Train/validation split

The real prediction task is forecasting dates the model has never trained on (validation.csv runs Nov-Dec 2025; the December chart needs Dec 2025), so a random K-fold split would overstate accuracy by letting the model interpolate between dates it has already seen on both sides. Instead, a time-based holdout was used: train on 38,477 rows from 2025-01-01 to 2025-08-31, evaluate on 9,523 rows from 2025-09-01 to 2025-10-31 (the most recent ~2 months, unseen during training). This mirrors the extrapolation the final model must do into November and December. A separate stress test additionally re-trained the model with 8 cities fully removed to confirm generalization to unseen cities specifically (Section 2.3).

### 3.3 Model comparison

Five models were compared on the same time-based holdout: Linear Regression, Random Forest, Gradient Boosting, XGBoost, and HistGradientBoostingRegressor. HistGradientBoostingRegressor with an absolute-error objective gave the best MAE and MAPE and was competitive on RMSE, and was selected as the final model (src/model.py). It also natively tolerates the mild outlier tail described in Section 2.5 better than a squared-error objective.

### 3.4 Final holdout performance

- MAE: \$118.82
- RMSE: \$636.85 (inflated by the unexplained-outlier tail, Section 2.5)
- MAPE: 5.20%
- R2: 0.826

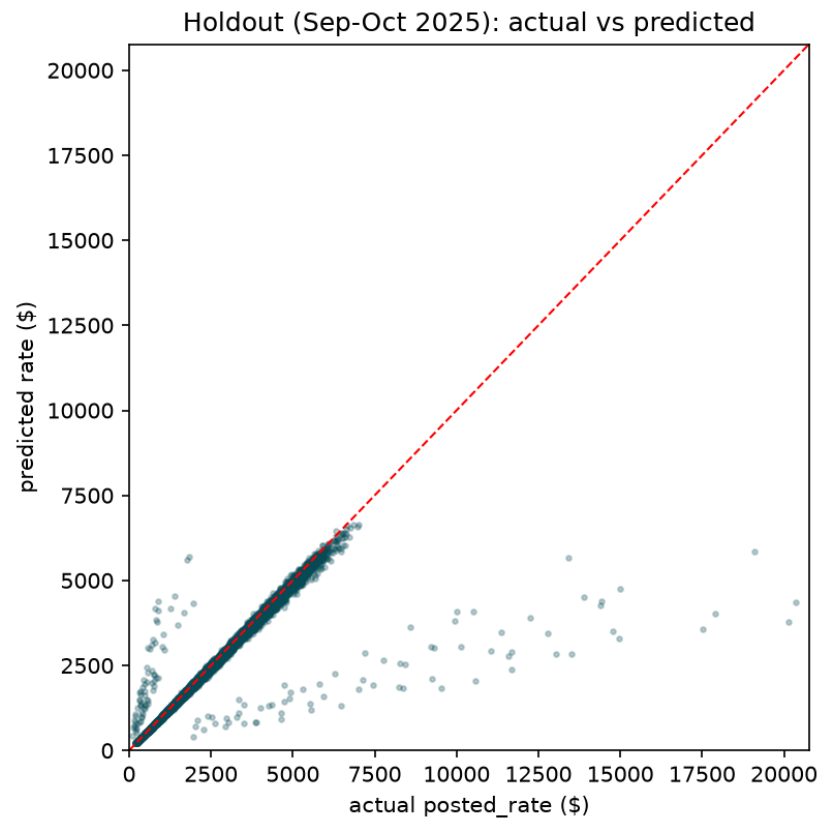


Figure 4. Holdout actual vs. predicted rate

## 4. Feature Importance

Permutation importance on the holdout set (drop in R2 when a feature is shuffled) confirms distance dominates the model, with equipment and weight contributing a small but non-zero amount. The geographic and date-cyclical features individually contribute very little once distance/equipment/weight are known - consistent with the weak raw correlations found in Section 1 - but they are what let the model generalize to unseen cities and produce a sensible December date trend (Section 5).

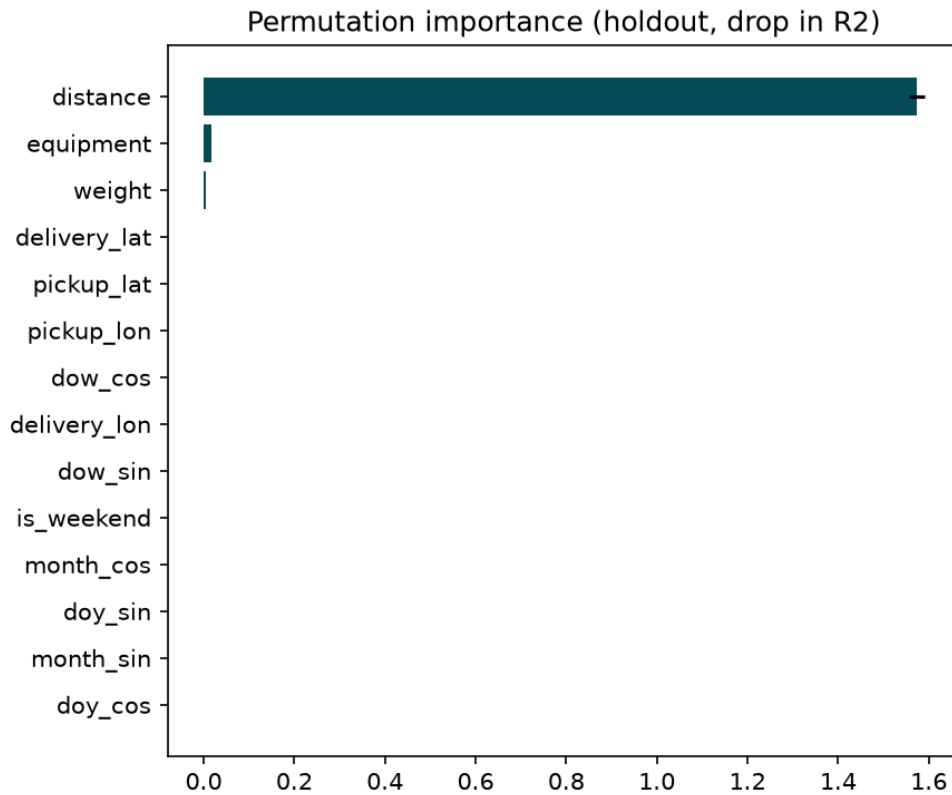


Figure 5. Permutation importance by feature

## 5. December 2025 Prediction Chart

The chart below is produced by the provided score.py from december\_predictions.csv (fixed route: Lexington -> Fort Wayne, 360 mi, Dry Van, 32,000 lb; only the date varies).

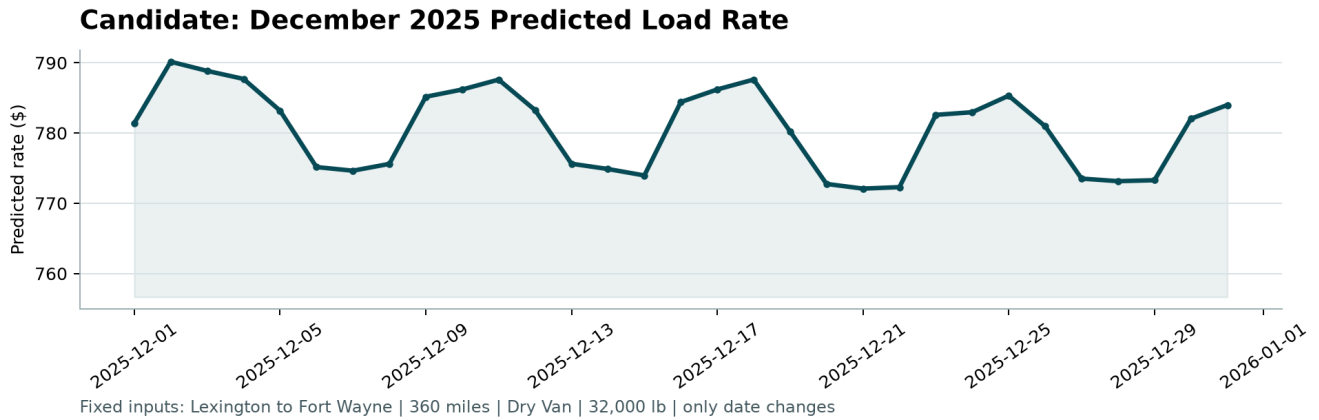


Figure 6. score.py output: scorer\_results/candidate\_december.png

Interpretation & caveat: because date-based features carry very little importance in the fitted model (Section 4), the predicted range across all 31 December days is narrow (about \$772-\$790, a ~2.3% spread) and shows a mild weekday-higher / weekend-lower pattern. This should be read as a small residual seasonal effect layered on a rate that is overwhelmingly set by distance, equipment and weight (all fixed for this route) - not as a strong, high-confidence seasonal forecast. Tree-based models also cannot extrapolate a linear trend beyond the training range; cyclical sin/cos date encodings were used specifically so that November/December values fall inside the numeric range already seen in training (see src/features.py), avoiding wild extrapolation, at the cost of not being able to project a trend that continues indefinitely upward or downward past October.

## 6. Reproducing These Results

From the repository root:

```
python -m pip install -r requirements.txt
python train.py          # cleans data, validates, fits final model, writes
                        # validation_predictions.csv and december_predictions.csv
python eda.py            # regenerates reports/figures used in this report
python score.py --predictions validation_predictions.csv \
                    --december-predictions december_predictions.csv
python generate_report.py # rebuilds this PDF
```

See README.md for a description of the repository layout (src/features.py, src/model.py, train.py, eda.py, score.py) and the full run instructions.